

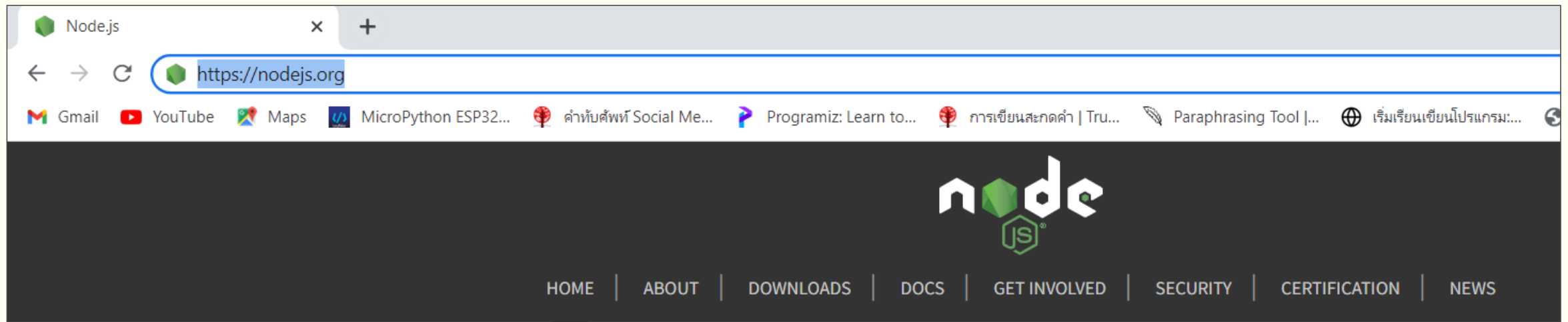


การติดตั้งและพื้นฐานของ REACT

030513702 : Selected Topics in Software Development
Asst. Prof. Dr. Phollakrit Wongsantisuk

การติดตั้ง Node.js

ถึงแม้ React จะเป็นการสร้าง Web Application สำหรับฝั่ง Frontend เป็นหลัก แต่โครงสร้างของแอปจะอยู่ในรูปแบบของ Node.js Application และจำเป็นต้องอาศัย Engine ของ Node.js ในการทำงาน



การติดตั้ง Node.js

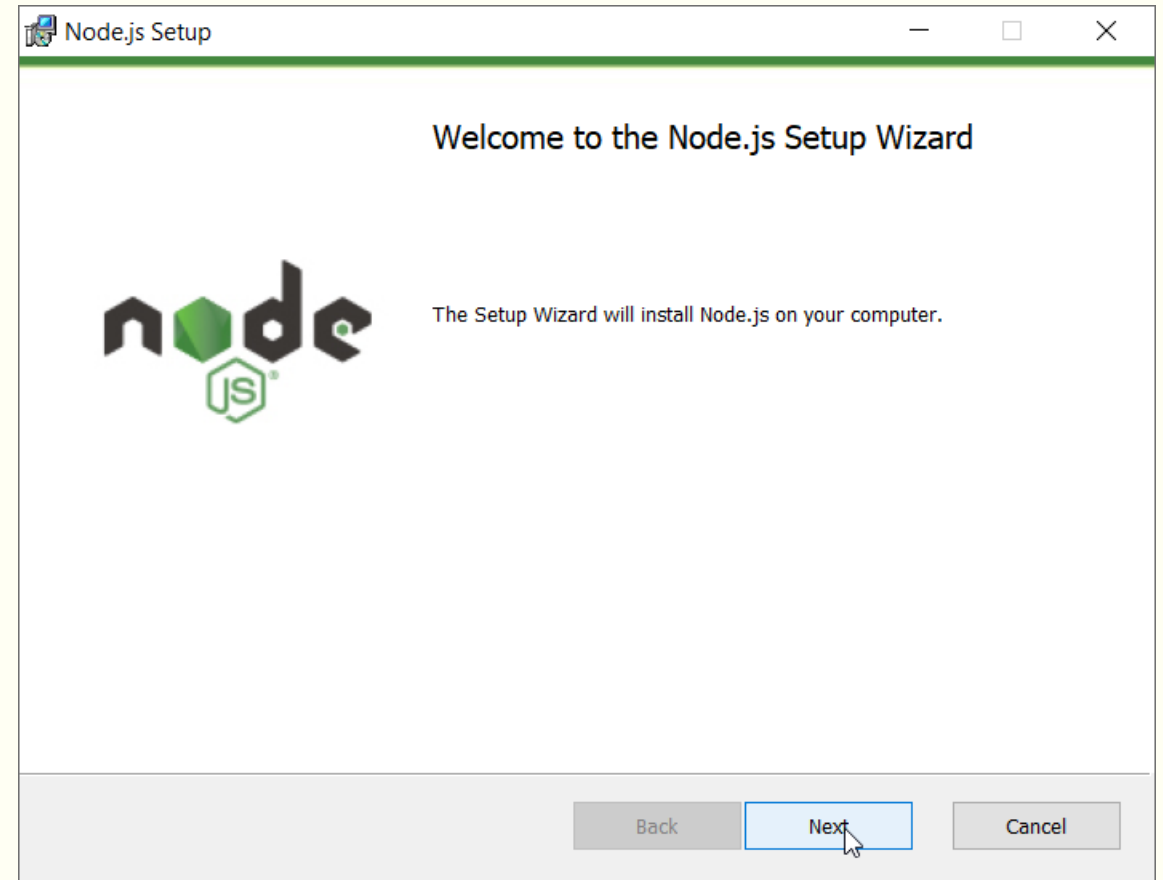
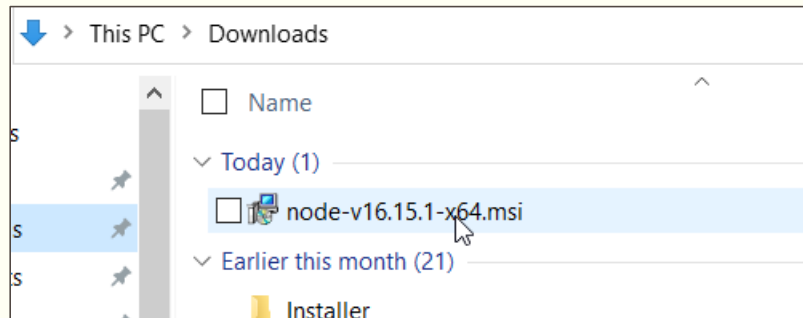
Download for Windows (x64)

16.15.1 LTS
Recommended For Most Users

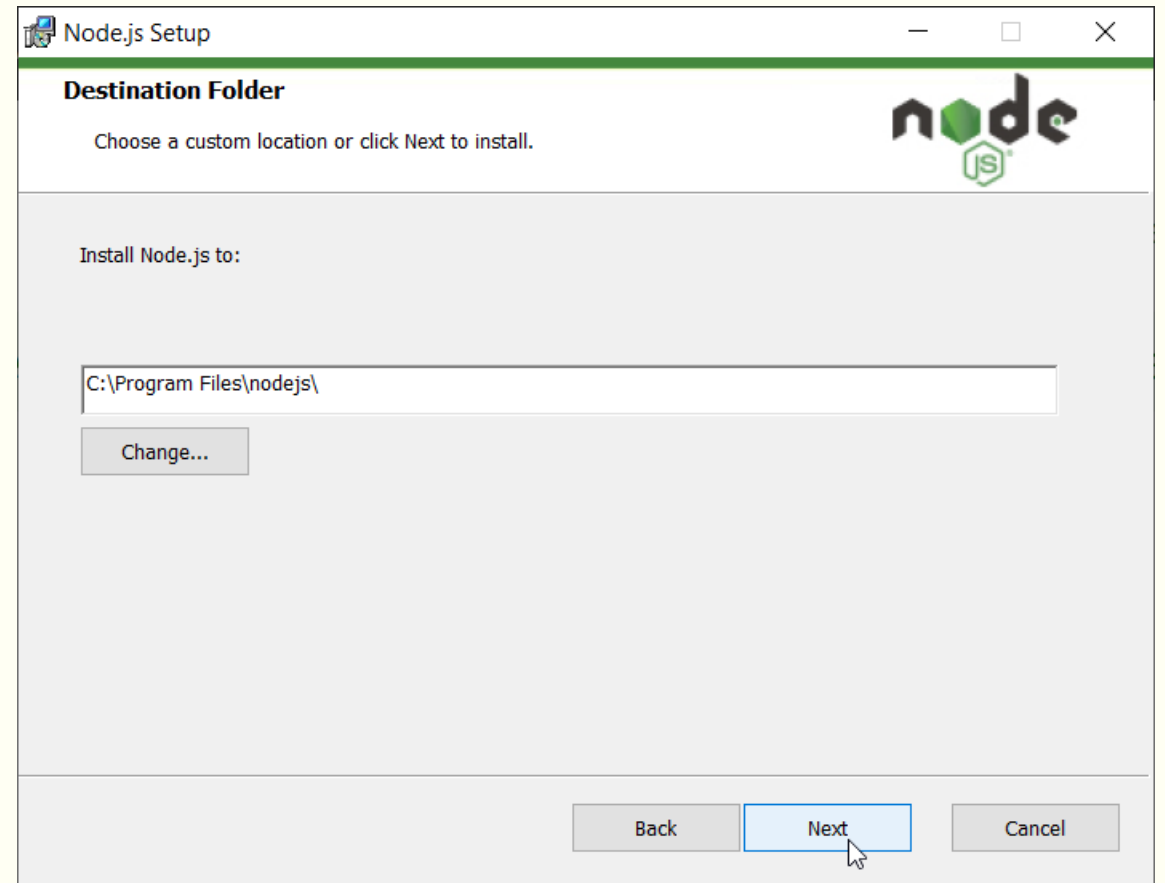
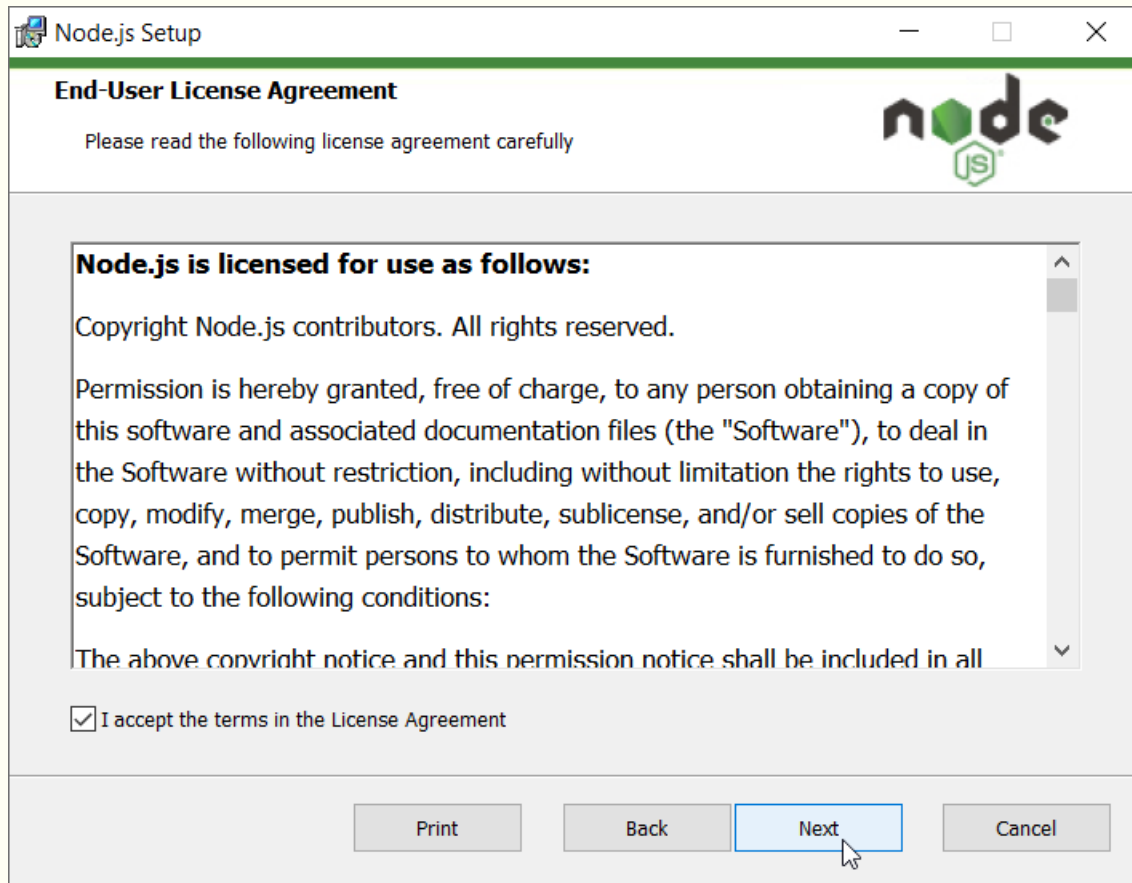
[Other Downloads](#) | [Changelog](#) | [API Docs](#)

18.4.0 Current
Latest Features

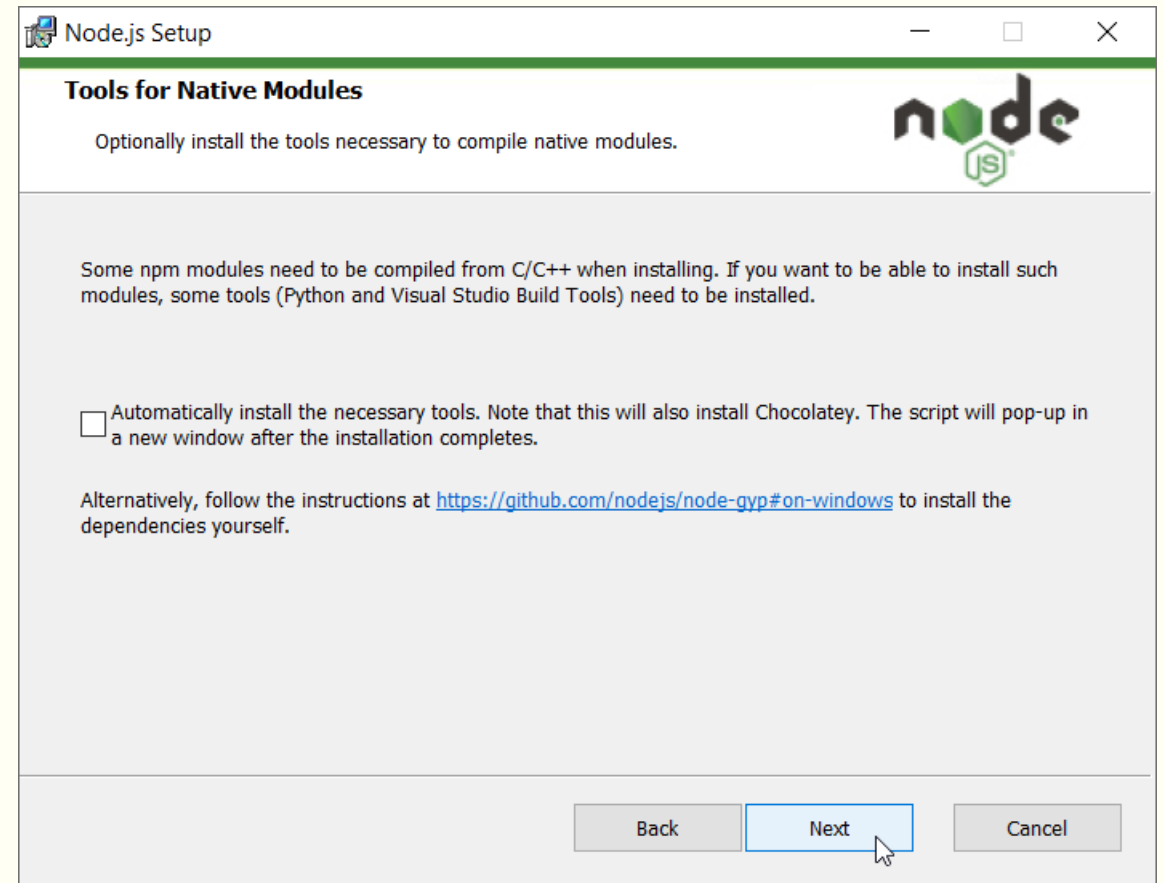
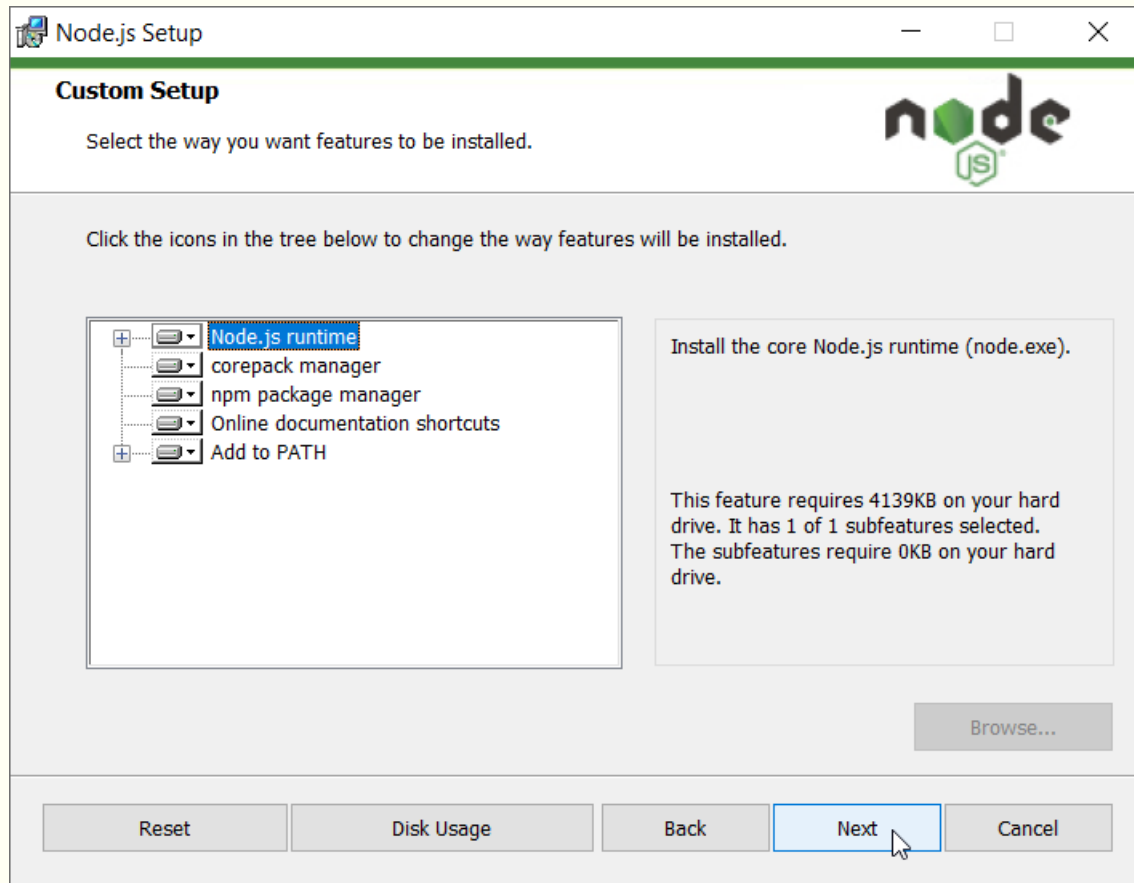
[Other Downloads](#) | [Changelog](#) | [API Docs](#)



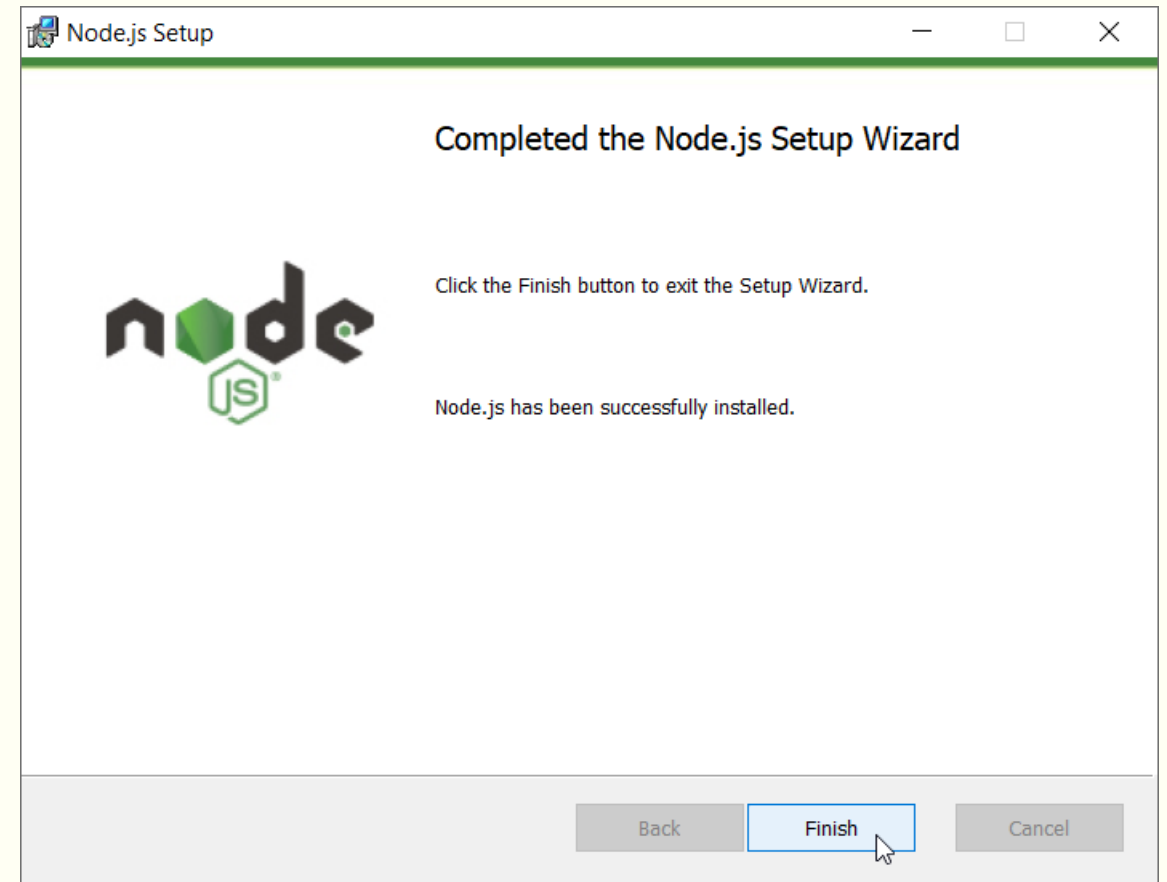
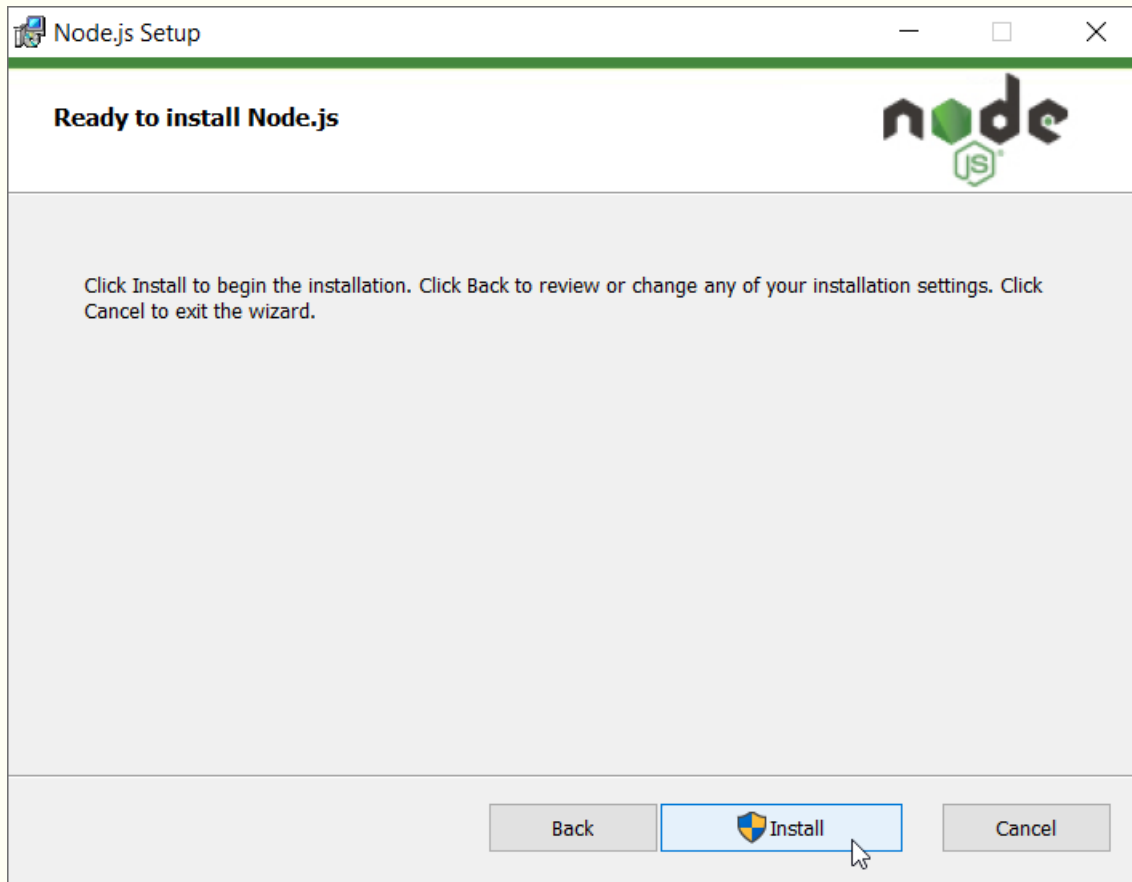
การติดตั้ง Node.js



การติดตั้ง Node.js



การติดตั้ง Node.js



การใช้ Terminal ของ VS Code

การทดสอบ React บน VS Code มีบางกรณีที่เราต้องพิมพ์คำสั่งผ่าน Command Line หรือ Terminal เพื่อดำเนินการต่างๆ เช่น การสร้างแอปของ React, การเข้าและออกจากไคลเอด์ทอรี, การติดตั้งโมดูล, การรันทดสอบแอป เป็นต้น และยังมีกรณีอื่นๆ อีกมากมายที่เราอาจต้องพิมพ์คำสั่งผ่าน Terminal ดังนั้น จึงควรมีพื้นฐานการเขียนคำสั่งผ่าน Terminal เอาไว้บ้าง

- คำสั่ง `cd` (หรือ `chdir`) คำสั่งเปลี่ยนไปยังไคลเอด์ทอรีที่ระบุ โดยกำหนดไคลเอด์ทอรีเป้าหมายตามคำสั่งนี้ เช่น

C:\Users\MY>cd d:\ebooks\javascript	ไปยังไคลเอด์ทอรี d:\ebooks\javascript
C:\Users\MY>cd documents\node	ไปยังไคลเอด์ทอรีที่อยู่ถัดจากตำแหน่งที่แสดง prompt คำสั่งในขณะนั้น ซึ่งกรณีนี้คือ c:\Users\MY ดังนั้น คำสั่งดังกล่าวจะไปที่ c:\Users\MY\documents\node
C:\Users\MY>cd ..	(2 จุด) ไปยังไคลเอด์ทอรีก่อนนี้ 1 ระดับ นั่นคือ c:\Users
C:\Users\MY>cd \	ไปยังไคลเอด์ทอรีเริ่มต้นที่แสดง prompt คำสั่งในขณะนั้น ซึ่งกรณีนี้จะไปที่ c:\

การใช้ Terminal ของ VS Code

- คำสั่ง md (หรือ mkdir) สำหรับสร้างไดเรกทอรีใหม่ โดยระบุชื่อไดเรกทอรีที่จะสร้างใหม่ตามหลังคำสั่งนี้

C:\Users\MY>mkdir js	สร้างไดเรกทอรี (โฟลเดอร์) ที่ชื่อ js ซ่อนไว้ในตำแหน่งที่แสดง prompt คำสั่งในขณะนั้น ซึ่งกรณีนี้จะสร้างไว้ที่ c:\Users\MY\js
C:\Users\MY>mkdir js\node	ถ้ายังไม่มีไดเรกทอรี js ก็จะสร้างขึ้นใหม่ แล้วสร้างไดเรกทอรี node ซ่อนไว้ใน js
C:\Users\MY>md d:\javascript\code	สร้างไดเรกทอรี javascript\code ไว้ที่ d:\

- คำสั่ง rm (หรือ rmdir) สำหรับลบไดเรกทอรี (ลบได้เฉพาะไดเรกทอรีว่าง) โดยระบุชื่อไดเรกทอรีที่ต้องการลบตามหลังคำสั่งนี้ แต่หากไดเรกทอรีนั้นไม่ว่าง เช่น มีไดเรกทอรีย่อยๆ และ/หรือ ไฟล์อยู่ข้างใน จะลบไม่ได้

C:\Users\MY>rmdir test	ลบไดเรกทอรี (โฟลเดอร์) ที่ชื่อ test ซึ่งซ่อนอยู่ในตำแหน่งที่แสดง prompt คำสั่งในขณะนั้น สำหรับกรณีนี้คือ c:\Users\MY\test
C:\Users\MY>rm d:\bad_dir	ลบไดเรกทอรี (โฟลเดอร์) ในตำแหน่งที่ระบุ (ถ้าว่าง)

การใช้ Terminal ของ VS Code

- คำสั่ง `del` (หรือ `erase`) สำหรับการลบไฟล์ โดยระบุตำแหน่งไฟล์ดังกล่าวตามหลังคำสั่งนี้ แต่หากกำหนดเป็นชื่อไดเรกทอรี จะหมายถึงการลบไฟล์ทั้งหมดในไดเรกทอรีนั้น

C:\Users\MY>del temp.docx	ลบไฟล์ temp.docx ออกจาก c:\Users\MY
C:\Users\MY>del d:\test\badfile.txt	ลบไฟล์ badfile.txt ออกจาก d:\test
C:\Users\MY>erase test1\test2	ลบไฟล์ทั้งหมดออกจากไดเรกทอรี c:\Users\MY\test1\test2 (แต่ไม่ลบไดเรกทอรี) และจะมี prompt ให้เรายืนยันด้วย

- คำสั่ง `ren` หรือ `rename` สำหรับเปลี่ยนชื่อไฟล์หรือไดเรกทอรี โดยระบุชื่อไฟล์หรือไดเรกทอรีเดิม แล้วตามด้วยชื่อใหม่ เช่น

C:\Users\MY>ren test.js demo.js	เปลี่ยนชื่อไฟล์ test.js ซึ่งอยู่ในตำแหน่งที่แสดง prompt คำสั่งในขณะนั้น สำหรับกรณีนี้คือ c:\Users\MY\test.js ไปเป็นชื่อ demo.js
C:\Users\MY>rename temp backup	เปลี่ยนชื่อไดเรกทอรี ซึ่งซ่อนอยู่ในตำแหน่งที่แสดง prompt คำสั่งในขณะนั้น สำหรับกรณีนี้คือ c:\Users\MY\temp ไปเป็นชื่อ backup

การใช้ Terminal ของ VS Code

- คำสั่ง `cls` สำหรับเคลียร์หน้าจอ โดยไม่จำเป็นต้องระบุพารามิเตอร์ใดๆ เช่น

C:\Users\MY>cls	เคลียร์ข้อความทั้งหมดบนหน้าจอของ Terminal
-----------------	---

- คำสั่ง `dir` สำหรับแสดงรายชื่อไฟล์และไดเรกทอรีย่อยๆ ที่อยู่ไดเรกทอรีของ prompt คำสั่งในขณะนั้น หรือที่ระบุตามหลังคำสั่งนี้ เช่น

C:\Users\MY>dir	แสดงรายชื่อไฟล์และไดเรกทอรีใน c:\Users\MY
C:\Users\MY>dir c:\	แสดงรายชื่อไฟล์และไดเรกทอรีใน c:\
C:\Users\MY>dir documents\js	แสดงรายชื่อไฟล์และไดเรกทอรีใน c:\Users\MY\documents\js
C:\Users\MY>dir d:\temp\download	แสดงรายชื่อไฟล์และไดเรกทอรีใน d:\temp\download

คำสั่ง NPM

หลังจากติดตั้ง Node.js เมื่อเปิดเข้าสู่โปรแกรม VS Code ก็สามารถใช้งาน Node.js ผ่าน Terminal ทันที ซึ่งโดยทั่วไปจะเป็นการพิมพ์ชุดคำสั่งที่เรียกว่า NPM (Node Package Manager) สำหรับคำสั่งที่ควรรู้จักในเบื้องต้นมีดังนี้

- `npm install <ชื่อแพ็คเกจ>` สำหรับการติดตั้งแพ็คเกจตามชื่อที่ระบุ
- `npm install -g <ชื่อแพ็คเกจ>` เมื่อเราต้องการติดตั้งแพ็คเกจแบบ global ให้สามารถใช้งานระหว่างแอปได้ โดยไม่จำเป็นต้องติดตั้งแบบซ้ำๆ ในแต่ละแอปที่จะใช้งาน
- `npm install <-g> <ชื่อแพ็คเกจ1> <ชื่อแพ็คเกจ2> <ชื่อแพ็คเกจ3> ...` เมื่อต้องการติดตั้งหลายๆแพ็คเกจพร้อมกัน (ไม่ต้องเสียเวลาติดตั้งทีละแพ็คเกจ)
- `npm uninstall <ชื่อแพ็คเกจ>` เมื่อต้องการลบหรือยกเลิกการติดตั้งแพ็คเกจตามชื่อระบุออกไปจากแอป

การใช้โมดูลของ JavaScript

โมดูล (Module) ก็คือไฟล์ที่จัดเก็บโค้ดของจาวาสคริปต์ ซึ่งอาจประกอบด้วย ตัวแปร ฟังก์ชัน คลาส ออบเจกต์ หรืออื่นๆ เพื่อนำไปใช้งานยังส่วนอื่นๆ และเนื่องจากการพัฒนาเว็บด้วย React นั้น จะเกี่ยวข้องกับการใช้โมดูลไปตลอด ดังนั้น เราควรรู้จักพื้นฐานในการนำเข้า (import) และส่งออก (export) โมดูลเอาไว้ล่วงหน้า

- **การสร้างและส่งออกโมดูล**

เนื่องจากโมดูลก็คือไฟล์จาวาสคริปต์ (.js) ดังนั้น หากจะสร้างโมดูล เราก็เริ่มจากการสร้างไฟล์ พร้อมกำหนดชื่อให้กับมัน (ชื่อไฟล์จะเป็นชื่อโมดูล) และให้มีส่วนขยายเป็น .js เช่น ในการทดสอบนี้ ผู้เขียนสร้างไฟล์ชื่อ math.js แสดงว่าโมดูลนี้ก็จะชื่อ math เป็นต้น และสมาชิกที่อยู่ในโมดูลอาจเป็น ตัวแปร ฟังก์ชัน คลาส ออบเจกต์ ก็ได้ สุดท้ายก็ส่งออกไปด้วยคำสั่ง export เช่น

การใช้โมดูลของ JavaScript

```
const PI = 3.141
```

```
function add(num1, num2) {  
    return num1 + num2  
}
```

```
class Random {  
    static getInteger(min, max) {  
        return min + Math.floor(Math.random() * (max - min))  
    }  
    static getBoolean() {  
        return (this.getInteger(1, 10) <= 5) ? true : false  
    }  
}
```

```
export {PI, add, Random} // ระบุสิ่งที่ส่งออกไปในวงเล็บ { }
```

//หรือเราอาจส่งออกในชื่อใหม่ โดยใช้คีย์เวิร์ด as เช่น

```
//export {PI, Random, add as plus}
```

//หมายถึง PI, Random ใช้ชื่อเดิมแต่ add เปลี่ยนเป็น plus

การใช้โมดูลของ JavaScript

หรืออีกวิธี นำคีย์เวิร์ด `export` มาวางไว้ด้านหน้าสมาชิกของโมดูลที่จะส่งออกไป ซึ่งเรียกว่า `inline export` เช่น

```
export const PI = 3.141

export function add(num1, num2) {
    ...
}

export class Random {
    ...
}
```

การใช้โมดูลของ JavaScript

- **การนำเข้าโมดูล**

การนำเข้าโมดูล จะใช้คำสั่ง import-form ดังรูปแบบต่อไปนี้

```
import {identifier} from <location>
```

- identifier ให้ระบุชื่อของสมาชิกที่ต้องการนำเข้า เช่น ชื่อตัวแปร ฟังก์ชัน คลาส หรือออบเจกต์ เป็นต้น โดยต้องเขียนไว้ในวงเล็บ { } เช่น {add, number, Random}
 - location คือ ตำแหน่งของโมดูล โดยเทียบกับตำแหน่งไฟล์ที่นำโมดูลนั้นมาใช้งาน ซึ่งการระบุตำแหน่งมีหลักการดังนี้
 - หากไฟล์ของโมดูลและไฟล์ที่ต้องการใช้โมดูลอยู่ในไดเรกทอรีเดียวกัน ให้แทนตำแหน่งด้วย './ชื่อไฟล์' เช่น './math' โดย . หมายถึงไดเรกทอรีปัจจุบัน
 - แต่ถ้าไฟล์ของโมดูลอยู่ในไดเรกทอรีย่อยๆ ลงไปอีก เราก็ต้องระบุตำแหน่งไปตามลำดับจนถึงไฟล์ของโมดูล เช่น import {...} from './modules/js/math'
- เช่น จากโมดูล math ที่เราได้สร้างไว้ในหัวข้อที่แล้ว หากเราต้องการนำเข้าฟังก์ชัน add, คลาส Random จากโมดูล math มาใช้งานในไฟล์ test-module.js ซึ่งเก็บอยู่ในโฟลเดอร์เดียวกับไฟล์โมดูล ก็อาจกำหนดโค้ดดังนี้

การใช้โมดูลของ JavaScript

```
import {add, Random} from './math'
```

```
alert(add(10, 20))
```

```
let r = Random.getInteger(1, 100)
```

- เราอาจตั้งชื่อใหม่ให้กับสมาชิกบางอันของโมดูลด้วย as แล้วเรียกใช้งานจากชื่อที่ตั้งขึ้นใหม่ เช่น

```
import {add as plus, Random} from './math'
```

```
let x = plus(1, 99)
```

- หากเราต้องการนำเข้าทุกอย่างที่มีในโมดูล สามารถแทนด้วยเครื่องหมาย * แล้วตั้งชื่อให้กับมันด้วย as ซึ่งกรณีนี้ไม่ต้องเขียนวงเล็บ {} จากนั้นเราอ้างอิงสมาชิกในโมดูลผ่านชื่อที่ตั้งขึ้น

```
import * as math from './math'
```

```
let x = math.add(10, 90)
```

```
let r = math.Random.getInteger(1, 100)
```

การใช้โมดูลของ JavaScript

- การส่งออกแบบ default

ภายในโมดูล หากมีสมาชิกเพียงอันเดียว เช่น คลาสเดียว หรือออบเจกต์เดียว หรือฟังก์ชันเดียว เราสามารถส่งออกสมาชิกแบบดีฟอลต์ โดยมีหลักการดังนี้

- หากใช้วิธี inline ก็ระบุคำว่า export default ที่สมาชิกตัวนั้น เช่น

```
export default class Test {  
    static hello() { }  
}
```

- หรืออาจกำหนดสมาชิกขึ้นมาก่อน แล้วใช้คำสั่ง export ที่หลัง เช่น

```
class Test() {  
    static hello() { }  
}  
  
export default Test //แบบที่ 1  
//export {Test as default} //แบบที่ 2
```

การใช้โมดูลของ JavaScript

- ขั้นตอนการนำเข้า ไม่ต้องมีวงเล็บ {} ครอบชื่อสมาชิกที่ส่งออกแบบ default ส่วนหลักการอื่นๆ ก็เหมือนเดิม เช่น

```
import Test from './test'
```

```
Test.hello()
```

- ถ้าต้องการนำเข้าโมดูลที่ส่งออกแบบ default แล้วเปลี่ยนชื่อ ก็ใช้ as ตามเดิม แต่กรณีนี้ต้องเขียนในวงเล็บ {} เช่น

```
import {default as demo} from './test'
```

```
demo.hello()
```

อย่างไรก็ตาม ไม่จำเป็นต้องใช้ default กับโมดูลที่มีสมาชิกเพียงอันเดียว เราสามารถใช้กับโมดูลที่มีสมาชิกมากกว่า 1 อันได้ แต่ในโมดูลเดียวกันจะมีสมาชิกแบบ default ได้เพียงอันเดียวเท่านั้น เช่น

```
//แบบที่ 1
```

```
export function hi() { return 'Hi' }
```

```
export default function hello() { return 'Hello' } //ส่งออกแบบ default
```

```
export function hey() { return 'Hey' }
```

การใช้โมดูลของ JavaScript

//แบบที่ 2

```
function hi() { return 'Hi' }
```

```
function hello() { return 'Hello' }
```

```
function hey() { return 'Hey' }
```

```
export { hi, hey, hello as default } //ส่งออก hello แบบ default
```

- แล้วในขั้นตอนการนำเข้า ก็กำหนดโค้ดแบบใดแบบหนึ่งในลักษณะดังนี้

//เอาตัว default ขึ้นก่อน

```
import hello, {hi, hey} from './example'
```

//หรือแบบอื่นๆ เช่น

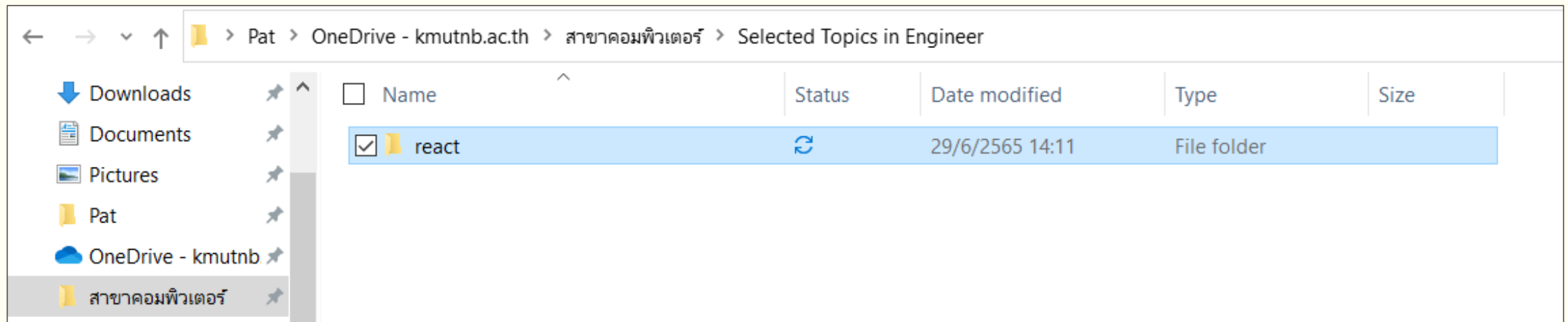
```
//import { default as hello, hi, hey } from './example'
```

```
//import hello, * as H from './example'
```

การสร้างและทดสอบ React Application

- สร้างโฟลเดอร์สำหรับจัดเก็บแอปต่างๆ

ให้สร้างโฟลเดอร์ขึ้นมา 1 โฟลเดอร์ เพื่อใช้เป็นที่เก็บแอปพลิเคชัน

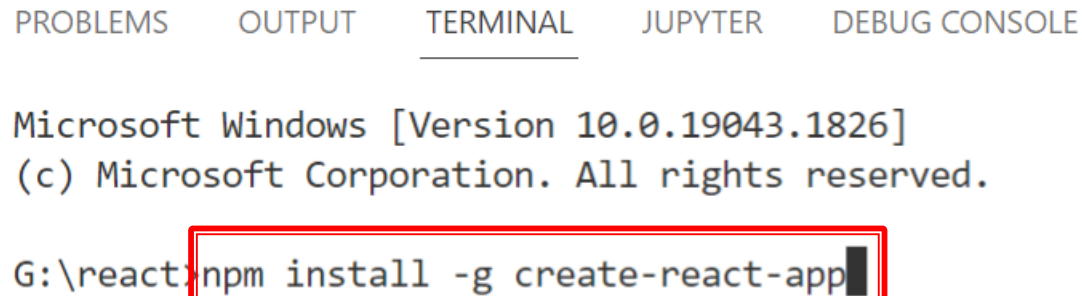


การสร้างและทดสอบ React Application

- การติดตั้งคำสั่ง create-react-app

เนื่องจากการสร้าง React App เราต้องพิมพ์คำสั่งในแบบของ npm แต่คำสั่งนี้ไม่ใช่องค์ประกอบพื้นฐานของ Node.js ดังนั้นเราจะต้องติดตั้งเพิ่มลงไปก่อน

```
npm install -g create-react-app
```



The screenshot shows a Windows terminal window with the following content:

```
PROBLEMS  OUTPUT  TERMINAL  JUPYTER  DEBUG CONSOLE

Microsoft Windows [Version 10.0.19043.1826]
(c) Microsoft Corporation. All rights reserved.

G:\react>npm install -g create-react-app
```

กรณีนี้ เราควรระบุ 옵션 -g ลงไป เพื่อให้คำสั่งหรือแพ็คเกจนี้ถูกติดตั้งแบบ Global แล้วในการสร้างแอปครั้งต่อไป ก็ไม่จำเป็นต้องติดตั้งคำสั่งนี้ซ้ำอีก แต่สามารถใช้งานได้ตลอด

การสร้างและทดสอบ React Application

```
G:\react>npm install -g create-react-app
npm WARN deprecated tar@2.2.2: This version of tar is no longer supported, and will not receive security updates. Please upgrade asap.

changed 67 packages, and audited 68 packages in 5s

4 packages are looking for funding
  run `npm fund` for details

2 high severity vulnerabilities

Some issues need review, and may require choosing
a different dependency.

Run `npm audit` for details.
```

หลังจากนั้นพิมพ์ `npm i tar`

```
G:\react>npm i tar

added 7 packages, and audited 8 packages in 2s

found 0 vulnerabilities

G:\react>
```

การสร้างและทดสอบ React Application

- การสร้าง React App

การสร้าง React App จะต้องใช้คำสั่ง create-react-app ตามที่เราได้ติดตั้งเอาไว้แล้ว ด้วยรูปแบบดังนี้

create-react-app <ชื่อแอป>

```
G:\react>npm i tar
added 7 packages, and audited 8 packages in 2s
found 0 vulnerabilities
G:\react>create-react-app app1
```

การสร้าง React App เราต้องเชื่อมต่ออินเทอร์เน็ตและใช้เวลาพอสมควรกว่าไฟล์ทั้งหมดจะถูกดาวน์โหลดมาติดตั้งจนครบสมบูรณ์

การสร้างและทดสอบ React Application

```
G:\react>create-react-app app1

Creating a new React app in G:\react\app1.

Installing packages. This might take a couple of minutes.
Installing react, react-dom, and react-scripts with cra-template...

added 1392 packages in 2m

204 packages are looking for funding
  run `npm fund` for details

Initialized a git repository.

Installing template dependencies using npm...
npm WARN deprecated source-map-resolve@0.6.0: See https://github.com/lydell/source-map-resolve#deprecated

added 52 packages in 11s

204 packages are looking for funding
  run `npm fund` for details
Removing template package using npm...

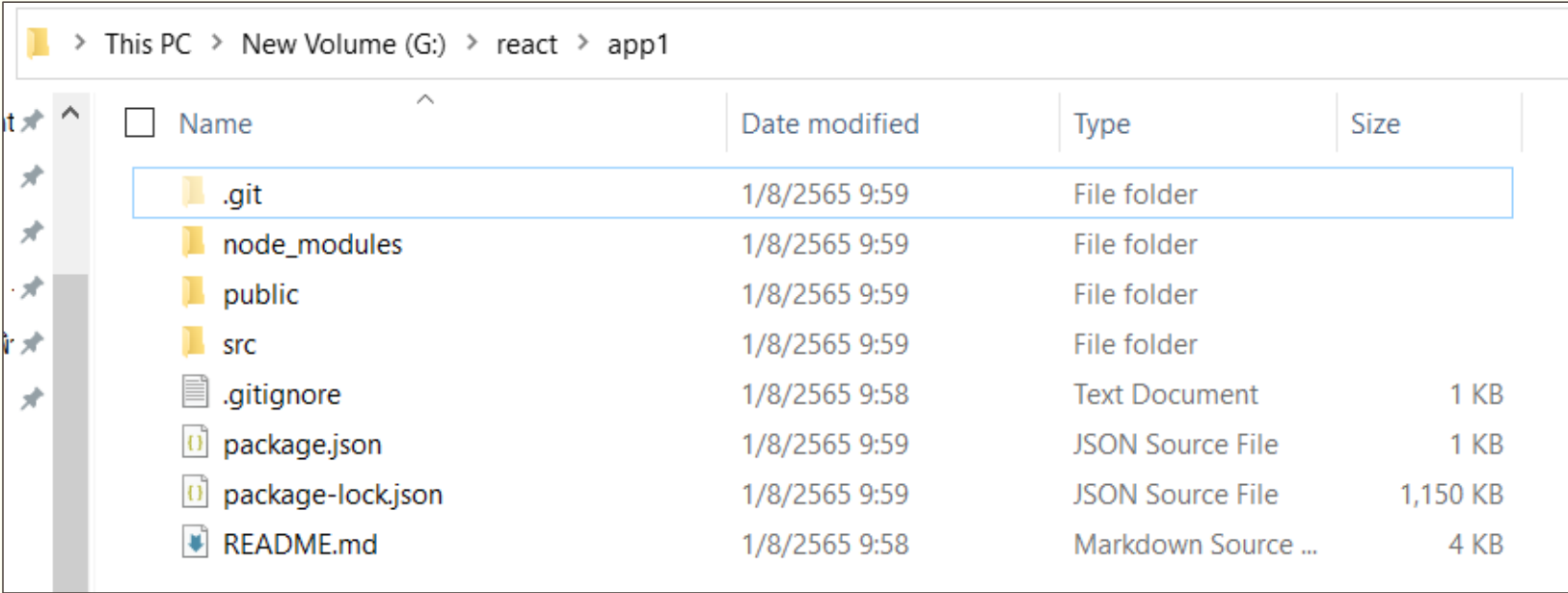
removed 1 package, and audited 1444 packages in 6s

204 packages are looking for funding
  run `npm fund` for details

6 high severity vulnerabilities

To address all issues (including breaking changes), run:
  npm audit fix --force
```

การสร้างและทดสอบ React Application



File Explorer window showing the contents of a directory named 'app1' inside a 'react' folder on a 'New Volume (G:)'. The directory contains several folders and files related to a React application setup.

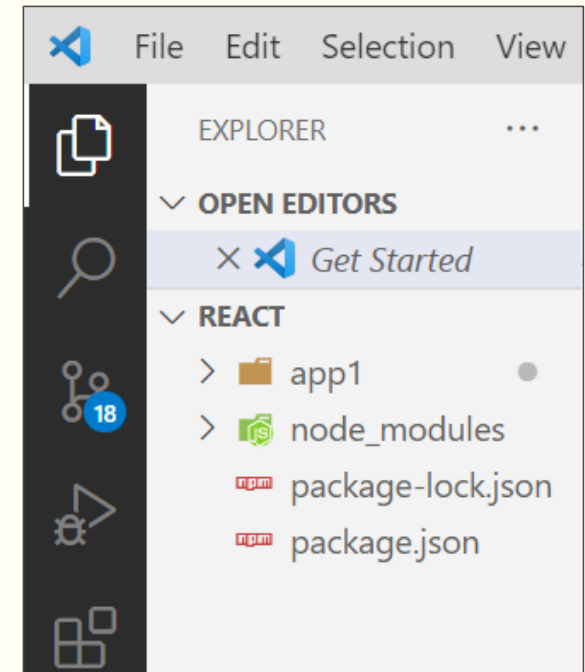
Name	Date modified	Type	Size
.git	1/8/2565 9:59	File folder	
node_modules	1/8/2565 9:59	File folder	
public	1/8/2565 9:59	File folder	
src	1/8/2565 9:59	File folder	
.gitignore	1/8/2565 9:58	Text Document	1 KB
package.json	1/8/2565 9:59	JSON Source File	1 KB
package-lock.json	1/8/2565 9:59	JSON Source File	1,150 KB
README.md	1/8/2565 9:58	Markdown Source ...	4 KB

การสร้างและทดสอบ React Application

- ลักษณะโครงสร้างของ React Application

โครงสร้างหรือองค์ประกอบภายในแอปจะแสดงดังภาพ สำหรับโฟลเดอร์และไฟล์ที่สำคัญภายในแอปที่ควรรู้จักในเบื้องต้นมีดังนี้

- **node_modules** เป็นโฟลเดอร์สำหรับจัดเก็บโมดูลที่จำเป็นสำหรับการทำงานของ React โดยโมดูลเหล่านี้จะอยู่ในรูปแบบของ Node.js หากเราติดตั้งโมดูลอื่นๆ เพิ่มเติมในภายหลังด้วยคำสั่ง npm ก็จะถูกนำมาเก็บไว้ในโฟลเดอร์นี้
- **public** เป็นโฟลเดอร์สำหรับจัดเก็บไฟล์ที่ต้องใช้งานร่วมกันระหว่างเพจหรือไฟล์อื่นๆ ซึ่งโดยส่วนใหญ่จะเป็นไฟล์แบบ Static หรือไฟล์ที่ไม่มีการเปลี่ยนแปลงเนื้อหา นั่นเอง เช่น ไฟล์ HTML, รูปภาพ เป็นต้น
- **src** เป็นโฟลเดอร์สำหรับจัดเก็บไฟล์ Source Code หลักของแอป โดยส่วนใหญ่จะเป็น JavaScript แต่บางกรณีก็อาจเป็น CSS โดยไฟล์สำคัญภายในโฟลเดอร์นี้คือ



การสร้างและทดสอบ React Application

- **src\App.css** เป็นที่จัดเก็บโค้ด CSS ส่วนกลาง สำหรับนำไปวางที่ส่วนอื่นๆ ในแบบ Placeholder CSS
- **src\App.js** เป็นไฟล์ของคอมโพเนนต์ระดับ Top-Level ของ React
- **src\index.js** คือไฟล์หลักสำหรับหน้าแรกของแอปพลิเคชัน หรือเทียบได้กับ Homepage ของเว็บไซต์นั่นเอง
- **src\index.css** เป็นไฟล์ CSS สำหรับหน้าแรก หรือใช้คู่กับไฟล์ index นั่นเอง
- **package.json** เป็นไฟล์ที่เก็บการตั้งค่าต่างๆ เกี่ยวกับแอปพลิเคชันนั้น
- **package-lock.json** ใช้จัดเก็บข้อมูลของโมดูลที่ติดตั้งในโฟลเดอร์ node_modules เพื่อใช้เป็นค่าอ้างอิงสำหรับคำสั่ง npm โดยทั่วไปเราไม่จำเป็นต้องจัดการใดๆ กับไฟล์นี้
- **.gitignore** รายชื่อไฟล์ที่ไม่ควรเพิ่มลงใน Git แต่ให้ใช้ภายในแอปนี้เท่านั้น
- **README.md** ส่วนขยาย .md คือ ไฟล์ประเภท Markdown และสำหรับไฟล์ README.md เป็นการแสดงคำสั่งและข้อมูลที่จำเป็นเมื่อเรานำแอปไปใช้บนแพลตฟอร์มอื่น เช่น GitHub

ทั้งหมดนี้เป็นเพียงองค์ประกอบพื้นฐานบางส่วนที่ React สร้างมาให้ นอกจากนี้เราอาจต้องสร้างไฟล์เพิ่มเติมหรือนำมาจากที่อื่นๆ เพื่อให้แอปสามารถทำงานได้ตามต้องการ

การสร้างและทดสอบ React Application

- การรันทดสอบแอป

หากต้องการรันทดสอบองค์ประกอบพื้นฐานที่ React สร้างมาให้ ต้องเปิดเข้าสู่แอปนั้นก่อน จากนั้นก็ทำตามขั้นตอนดังนี้

1. ที่ Terminal ต้องเข้าไปในโฟลเดอร์ของแอปที่ต้องการรัน เช่น สมมติว่าต้องการรันแอป app1-1 ก็ใช้คำสั่ง cd เพื่อเข้าไปยังโฟลเดอร์ดังกล่าว

```
PROBLEMS  OUTPUT  TERMINAL  JUPYTER  GITLENS  DEBUG CONSOLE

Microsoft Windows [Version 10.0.19043.1826]
(c) Microsoft Corporation. All rights reserved.

G:\react>cd app1
G:\react\app1>
```

2. พิมพ์คำสั่ง **npm start** ลงบน Terminal

```
G:\react\app1>npm start

> app1@0.1.0 start
> react-scripts start
```

การสร้างและทดสอบ React Application

3. ระบบเซิร์ฟเวอร์จำลองของ React จะเริ่มทำงาน หลังจากนั้นเว็บเบราว์เซอร์ที่ถูกตั้งค่าเป็นดีฟอลต์จะถูกเปิดขึ้นมาโดยอัตโนมัติพร้อมกับแสดงผลที่ตำแหน่ง <http://localhost:3000> ดังภาพ ซึ่งเป็นหน้า Homepage ที่ React สร้างไว้ให้

ระบบเซิร์ฟเวอร์จำลองจะทำงานต่อเนื่องไปเรื่อยๆ ซึ่งในระหว่างนั้น หากเราเปิดเบราว์เซอร์ตัวอื่นขึ้นมาแล้วกำหนด URL เป็น <http://localhost:3000> ก็จะได้ผลในแบบเดียวกัน และถ้าต้องการหยุดการรัน ให้ไปที่ Terminal แล้วกด **<Ctrl+C>** จากนั้นเมื่อมี Prompt ขึ้นมาถาม ให้ตอบ Y (Yes) เพื่อยืนยัน แล้วเซิร์ฟเวอร์จำลองจะหยุดทำงาน

```
PROBLEMS  OUTPUT  TERMINAL  JUPYTER  GITLENS  DEBUG CONSOLE

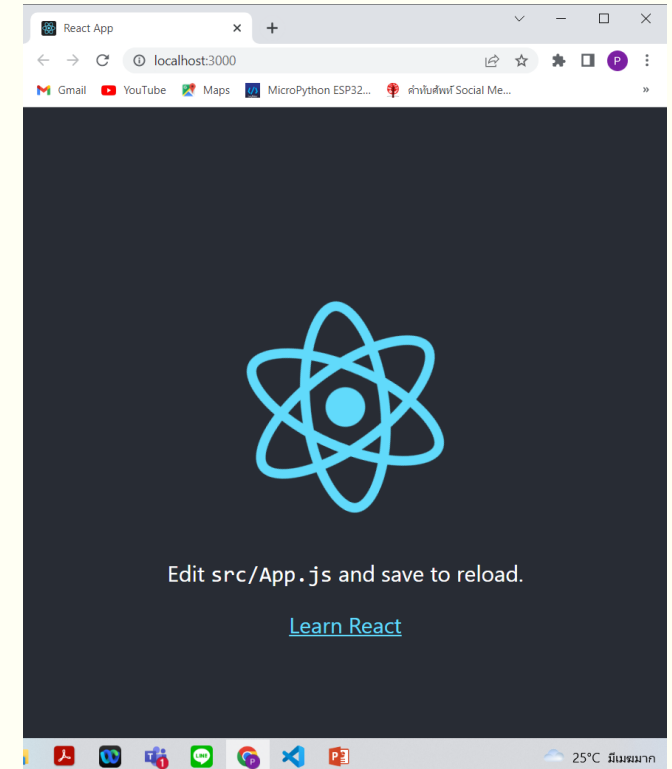
Compiled successfully!

You can now view app1 in the browser.

Local:      http://localhost:3000
On Your Network:  http://10.62.62.84:3000

Note that the development build is not optimized.
To create a production build, use npm run build.

webpack compiled successfully
Terminate batch job (Y/N)? y
```



หลักการของ React JSX

JSX (JavaScript XML) เป็นเทคนิคการเขียนแท็ก HTML ภายในโค้ดของ JavaScript โดยไม่ใช้วิธีการแบบสตริง ซึ่ง JSX ถือเป็นจุดเด่นอีกอย่างหนึ่งของ React

เราลองมาดูวิธีการเขียน HTML ลงในโค้ด JavaScript แบบปกติทั่วไป ที่ต้องกำหนดขอบเขตด้วยเครื่องหมายคำพูด (Quotes หรือ Backticks) อันใดอันหนึ่ง ดังนี้ (ยังไม่ใช้ JSX)

```
const head = "<h2>React & React Native</h2>"
const text = '<div><b>Create Web & Mobile Apps with  
JavaScript</b></div>'
const table=`
  <table>
    <tr><th>Column 1</th><th>Column 2</th></tr>
    <tr><td>...</td><td>...</td></tr>
    ...
  </table>`
```

หลักการของ React JSX

แต่การเขียนในแบบ JSX เราไม่ต้องเขียนเครื่องหมายคำพูดกำกับ สามารถเขียนโค้ด HTML ลงไปโดยตรง เหมือนกับในเว็บเพจ และถ้าสตริงนั้นยาวเกิน 1 บรรทัด ให้เขียนไว้ในวงเล็บ () เช่น

```
const head = <h2>React & React Native</h2>
const text = <div><b>Create Web & Mobile Apps with JavaScript</b></div>
const table=(
  <table>
    <tr><th>Column 1</th><th>Column 2</th></tr>
    <tr><td>...</td><td>...</td></tr>
    ...
  </table>
)
```

โค้ดดังกล่าวไม่ตรงตามหลักการของ JavaScript ดังนั้น React จึงมีกลไกที่ช่วยแปลงจาก JSX ไปเป็นสตริงตามรูปแบบที่ถูกต้องของ JavaScript ดังนั้น จึงสามารถนำไปแสดงผลบนเบราว์เซอร์ได้ตามปกติ โดยไม่เกิดปัญหาใดๆ



หลักการของ React JSX

การใช้ JSX ยังมีข้อกำหนดและลักษณะที่ควรรู้เพิ่มเติมอีกหลายประการ ดังรายละเอียดต่อไปนี้

- ไฟล์ JavaScript ที่เราจะเขียน JSX ได้ ต้องนำเข้า (import) คอมโพเนนต์ที่ชื่อ React จากโมดูล 'react' เป็นอันดับแรก เช่น

```
import React from 'react'
```

- แท็ก HTML ที่เขียนในแบบ JSX ต้องบรรจุอยู่ใน Parent หรือมี Container ที่มีแท็กเปิดและแท็กปิด เพื่อกำหนดขอบเขตแทนการใช้เครื่องหมายคำพูด เช่น

```
const t1 = <div><b>Hello React</b></div>      // มี div เป็น Parent
const t2 = <>This text is in empty parent</>  // สามารถใช้แท็กว่างเป็น Parent ก็ได้

// แต่ถ้าเขียนแบบนี้จะเกิดข้อผิดพลาด เพราะทั้งหมดไม่อยู่ใน Parent
const t3 = <br/><span>Hi React</span>           // Error เพราะเริ่มต้นด้วย <br/>
const t4 = สร้าง Mobile App ด้วย React Native // Error เพราะไม่มี Parent
```

หลักการของ React JSX

- เนื่องจาก JSX ใช้หลักการของ XML ดังนั้น แท็ก HTML ทั้งหมดที่เขียนต้องมีทั้งแท็กเปิดและปิดคู่กันเสมอ คือเริ่มต้นด้วย Open Tag ในแบบ <tagname> และสิ้นสุดด้วย Close Tag ในแบบ </tagname> แต่ในกรณีที่เป็นแท็กเดี่ยว (Single Tag) ให้เขียนในแบบ <tagname/> เช่น
- เราสามารถเขียน HTML ในแบบ JSX ให้กับตัวแปร ค่าคงที่ ฟังก์ชัน หรือกำหนดเป็นอาร์กิวเมนต์ของฟังก์ชัน และอื่นๆ อีกหลายแบบ เช่น

แท็กปกติของ HTML	แท็กในแบบ JSX
<hr>	
<input ...>	<input .../>
	

```
import React from 'react'

let title = <h2>React & React Native</h2>

function showFooter() {
  return <div>&copy;2000-2030 developerthai.com</div>
}
```

```
function test(html) {
  ...
}

test(<h3>Hello</h3>) // สามารถกำหนด HTML ให้เป็น
                    // อาร์กิวเมนต์เมื่อเรียกฟังก์ชัน
```

หลักการของ React JSX

- เราสามารถแทรกค่าตัวแปรหรือค่าจากฟังก์ชัน หรือ Expression ที่มีผลลัพธ์เกิดขึ้น ลงใน JSX ได้เช่นเดียวกับการแทรกลงใน Template String ของ JavaScript โดยระบุตัวแปรไว้ในวงเล็บ {} แต่ในกรณีนี้ไม่ต้องเขียนเครื่องหมาย \$ ไว้หน้าวงเล็บเหมือนที่ต้องทำใน Template String ของ JavaScript เช่น

```
import React from 'react'

let r = 'React'
let rn = 'React Native'
let title = <span>Developing Web & Mobile Apps with {r} & {rn}</span>
let a = 10
let b = 20
let x = <div>{a} + {b} = {a + b}</div>
```

หลักการของ React JSX

- หากข้อมูลที่นำมาแทรกเป็นแบบอาร์เรย์ สมาชิกทั้งหมดจะถูกเชื่อมต่อเป็นสตริงเดียวกัน เช่น

```
let a = ['red', 'green', 'blue']  
let b = <div>{a}</div>           //b = redgreenblue
```

- หากตัวแปรที่เรานำมาแทรก มีแท็ก HTML รวมอยู่ด้วย (HTML String) เมื่อแสดงผล จะปรากฏเป็นแท็กในแบบสตริงตามเดิม เช่น `<b>Hello</b>` แต่หากเราต้องการให้แท็กถูกประมวล ต้องกำหนดด้วยแอตทริบิวต์ `dangerouslySetInnerHTML` ดังโค้ดต่อไปนี้

```
let r = '<b>React</b>'  
let a = <span>{r}</span>           //a = <b>React</b>  
let b = <span dangerouslySetInnerHTML={{__html: r}}/> //React จะเป็นตัวหนา
```

หลักการของ React JSX

- นอกจากการแทรกค่าตัวแปรแล้ว เรายังสามารถกำหนดเงื่อนไขแบบ Ternary ภายในวงเล็บ {} ดังแนวทางต่อไปนี้

```
let success = false
const msg = <div>{(success) ? 'Yes' : 'No'}</div>
//หมายความว่า หาก success = true จะแทรกคำว่า 'Yes' มิฉะนั้นจะแทรกคำว่า 'No'
```

```
let num = parseInt(Math.random()*10)
const oddEven = (
  <div>
    {(num % 2 == 0) ? 'Even' : 'Odd'}
  </div>
)
//ถ้าหาร 2 ลงตัว จะแทรกคำว่า 'Even' มิฉะนั้นจะแทรกคำว่า 'Odd'
```

หลักการของ React JSX

- เนื่องจาก HTML ที่เขียนในแบบ JSX ต้องถูกแปลงเป็นสตริงในรูปแบบโค้ดของ JavaScript ดังนั้น การเขียนแอตทริบิวต์ของ HTML ต้องตรงตามหลักการของ JavaScript ซึ่งสามารถสรุปประเด็นที่น่าสนใจได้ดังนี้
 - แอตทริบิวต์ที่ตรงกับ Reserved Word ของ JavaScript ได้แก่ class และ for ให้แทนด้วยค่าใหม่ ดังนี้

แอตทริบิวต์แบบ HTML	แอตทริบิวต์แบบ JSX
class	className
for (ปกติใช้กับ label)	htmlFor

```
<div className="container">...</div>
<label htmlFor="login">Login</label>
<input id="login" .../>
```

- แอตทริบิวต์ที่เป็นคำเดียว ให้เขียนเป็นตัวพิมพ์เล็กทั้งหมดตามแบบเดิมของ HTML เช่น id, style, name, value, placeholder, src, disabled เป็นต้น

หลักการของ React JSX

- แอตทริบิวต์ที่เกิดจากการนำตั้งแต่ 2 คำขึ้นไปมารวมกัน ให้เขียนแบบ camelCase นั่นคือ คำแรกให้ขึ้นต้นด้วยตัวพิมพ์เล็ก แต่คำที่ 2 เป็นต้นไปให้ขึ้นต้นด้วยตัวพิมพ์ใหญ่ เช่น

แอตทริบิวต์แบบ HTML	แอตทริบิวต์แบบ JSX
maxlength	maxLength
tabindex	tabIndex
onclick	onClick
onmousedown	onMouseDown
...	...

```
const input = <input type="number" id="n1" className="num" maxLength="5" />
```

หลักการของ React JSX

- หากต้องการเขียนคำอธิบาย (Comment) แทรกใน JSX ต้องเขียนในแบบ `{/* คำอธิบาย */}` เท่านั้น จะใช้รูปแบบของ HTML คือ `<!-- -->` ไม่ได้ เช่น

```
const table = (  
  <table>  
    <tr><th>Product</th><th>Price</th></tr>  
    /*  
    <tr><td>React</td><td>1000</td></tr>  
    */  
  </table>  
)
```

ReactDOM และการแสดงเว็บเพจ

ในมุมมองของ JavaScript นั้น DOM (Document Object Model) คือ ออบเจกต์สำหรับการเข้าถึงและจัดการอีลิเมนต์ต่างๆ ภายในเพจ เช่น อ่านค่า เปลี่ยนแปลง เพิ่ม และลบอีลิเมนต์ในเอกสาร HTML เป็นต้น ส่วนใน React ก็มี DOM เช่นเดียวกัน โดยอยู่ในรูปแบบของคอมโพเนนต์ที่ชื่อ ReactDOM ซึ่งหน้าที่หลักๆ ที่เราจะได้ใช้งานในเบื้องต้นก็คือ การแสดงผลเว็บเพจ โดยมีแนวทางดังต่อไปนี้

- การใช้งาน ReactDOM อาจทำได้ใน 2 รูปแบบ คือ
 - **แบบที่ 1** นำเข้าคอมโพเนนต์ ReactDOM จากโมดูล 'react-dom' แล้วแสดงผลโดยเรียกเมธอด render() ผ่านคอมโพเนนต์นี้ เช่น

```
import ReactDOM from 'react-dom'
```

```
ReactDOM.render(...) //เมื่อต้องการแสดงผล
```

ReactDOM และการแสดงเว็บเพจ

- **แบบที่ 2** นำเข้าเมธอด render() แบบเจาะจงจากโมดูล 'react-dom' แล้วแสดงผลด้วยการเรียกเมธอด render() โดยตรง เช่น

```
import { render } from 'react-dom'

render(...) //เมื่อต้องการแสดงผล
```

- เมธอด render() ซึ่งใช้สำหรับการแสดงผลเว็บเพจ มีรูปแบบดังนี้

```
render(element, container [, callback])
```

- element คือ สิ่งที่เราจะแสดงผล ซึ่งอาจกำหนดเป็น HTML ในแบบ JSX โดยตรง หรือนำค่ามาจากตัวแปร หรือฟังก์ชัน หรืออื่นๆ อีกหลายแบบ
- container คือ เป้าหมายในการแสดงผล (นำ HTML มาแสดงผลในอิลิเมนต์นี้)
- callback อาจระบุเมื่อเราต้องการกระทำบางอย่างเมื่อแสดงผล โดยเรียกฟังก์ชันเป้าหมายขึ้นมาทำงาน

ReactDOM และการแสดงเว็บเพจ

- สำหรับแนวทางการนำไปใช้ก็คือ การแสดงผลแอปที่เราสร้างขึ้น โดยในเบื้องต้นมี 3 ไฟล์หลักๆ ที่เกี่ยวข้องคือ
 - `public/index.html` เป็นตำแหน่ง container ที่จะแสดงผล
 - `src/App.js` กำหนดองค์ประกอบส่วนต่างๆ ของแอป
 - `src/index.js` แสดงองค์ประกอบที่กำหนดไว้ใน `App.js` ออกไปที่ `index.html`

สำหรับในเบื้องต้น เราจะเขียนโค้ดที่ไฟล์ `App.js` เป็นหลัก โดยหากต้องการทดสอบ JSX ก็อาจกำหนดโค้ดไว้ในฟังก์ชัน `App()` โดยลบโค้ดเดิมที่ React สร้างไว้ให้ในฟังก์ชันทิ้งไป โดยหลังการแก้ไขโค้ด ให้บันทึกการเปลี่ยนแปลงก่อนรันทดสอบ (`Ctrl + K + S`) แล้วทำการ Refresh เบราเซอร์เพื่อดูผลลัพธ์ล่าสุด ซึ่งแนวทางการทดสอบบางส่วนเป็นดังนี้ (นำมาเฉพาะโค้ดที่เกี่ยวข้องในไฟล์ `App.js`)

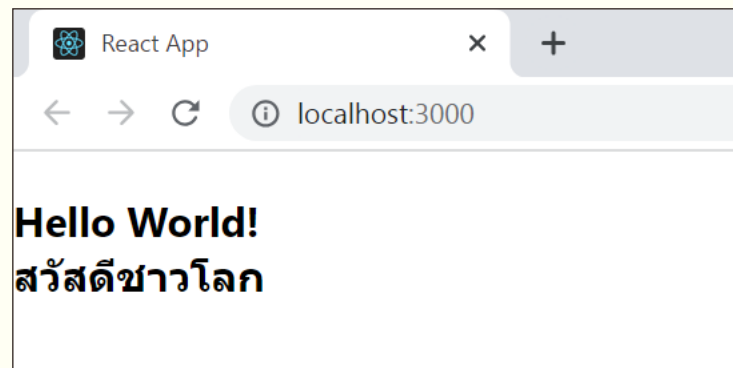
ReactDOM และการแสดงเว็บเพจ

react/app1/src/App.js

```
import React from 'react'

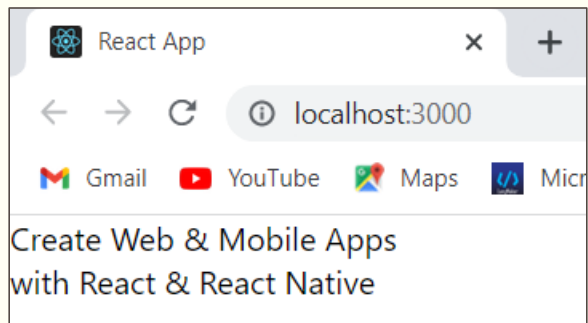
function App() {
  return (
    <h3>Hello World! <br/>สวัสดีชาวโลก</h3>
  )
}

export default App;
```

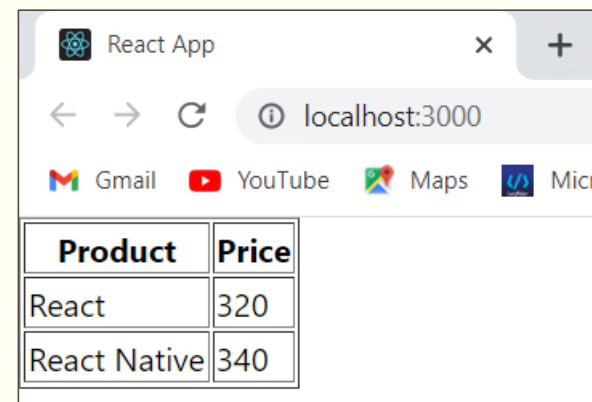


ReactDOM และการแสดงเว็บเพจ

```
function App() {  
  let r = 'React'  
  let rn = 'React Native'  
  const el = (  
    <div>  
      Create Web & Mobile Apps<br/>  
      with {r} & {rn}  
    </div>  
  )  
  return el  
}
```



```
function App() {  
  return (  
    <table border="1">  
      <tr><th>Product</th><th>Price</th></tr>  
      <tr><td>React</td><td>320</td></tr>  
      <tr><td>React Native</td><td>340</td></tr>  
    </table>  
  )  
}
```



การกำหนดสไตล์ CSS ร่วมกับ JSX

การปรับแต่งเว็บเพจด้วย CSS หากเขียนโค้ดในแบบ JSX ก็มีหลักการบางส่วนที่แตกต่างไปจากกรณีที่เรากำหนด CSS ให้กับเอกสาร HTML โดยตรง ซึ่งการนำ CSS มาใช้ร่วมกับ JSX จะแบ่งเป็น 2 ลักษณะคือ แบบ Inline Style และ External Style ดังรายละเอียดต่อไปนี้

- **การกำหนดสไตล์แบบ Inline**

การกำหนดสไตล์แบบ Inline ก็คือการเขียนโค้ด CSS ไว้ที่แท็ก HTML โดยตรง ซึ่งในกรณีที่เราเขียนโค้ดในแบบ JSX ก็มีหลักการที่แตกต่างจากแบบปกติดังนี้

- กำหนดสไตล์ในแบบออบเจกต์ของ JavaScript โดยชื่อพรีอเพอร์ตี้ต้องเขียนในแบบ camelCase ห้ามมีขีดคั่นระหว่างคำตามหลักการของ JavaScript และค่าของพรีอเพอร์ตี้ต้องกำหนดไว้ในเครื่องหมายคำพูด
- นำออบเจกต์ของสไตล์มากำหนดให้กับแท็กด้วยแอตทริบิวต์ style ตามปกติ แต่ต้องเขียนค่าของมันไว้ในวงเล็บ {}

เช่น

```
const divStyle = {                                //สร้างสไตล์ในแบบออบเจกต์
  color: 'red',
  backgroundColor: 'powderblue', //แบบ CSS คือ background-color
  fontSize: 'larger',           //แบบ CSS คือ font-size
}
return <div style={divStyle}>Hello React</div>
```

การกำหนดสไตล์ CSS ร่วมกับ JSX

- เราสามารถกำหนดสไตล์ย่อยๆ ให้แก่แท็กเดียวกันในแบบของอาร์เรย์ เช่น

```
const redText = { color: 'red' }  
const darkBg = { backgroundColor: 'black' }  
...  
return <div style={ [redText, darkBg] }>Hello React</div>
```

- หรืออีกวิธีคือ กำหนดสไตล์ในแบบออบเจกต์ให้กับแท็กโดยตรง ดังโค้ดถัดไป (วงเล็บ {} อันข้างในใช้กำหนดออบเจกต์ ส่วนอันข้างนอกใช้สำหรับกำหนดสไตล์ในแบบ JSX)

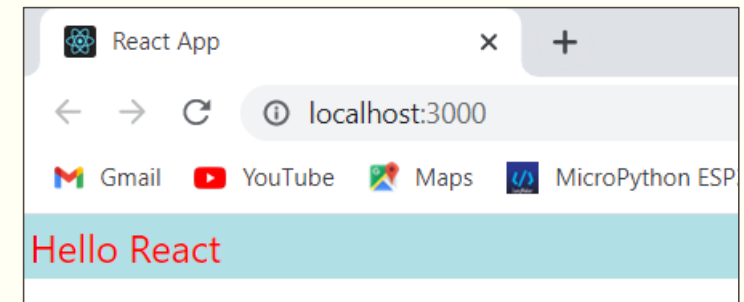
```
<div style={ {color: 'red', backgroundColor: 'powerblue'} }>  
  Hello React  
</div>
```

การกำหนดสไตล์ CSS ร่วมกับ JSX

แนวทางการเขียนโค้ดโดยรวมที่ไฟล์ App.js จะมีลักษณะดังนี้

react/app1/src/App.js

```
function App() {  
  const divStyle = {  
    color: 'red',  
    backgroundColor: 'powderblue',  
    fontSize: 'larger',  
    padding: '3px',  
  }  
  return <div style={divStyle}>Hello React</div>  
}
```



การกำหนดสไตล์ CSS ร่วมกับ JSX

- **การกำหนดสไตล์แบบ External**

การกำหนดสไตล์แบบ External เป็นการแยกส่วนของ CSS ไปสร้างเป็นไฟล์ไว้ต่างหาก และในกรณีของ React จะนำสไตล์เข้ามาใช้งานในเพจหรือไฟล์ที่ต้องการ โดยใช้คำสั่ง `import` ในแบบเดียวกับโมดูล ซึ่งหลักการที่สำคัญมีดังนี้

- สร้างไฟล์ใหม่เพิ่มเข้ามาในแอป โดยให้มีส่วนขยายเป็น `.css` แล้วภายในไฟล์ดังกล่าว จะต้องมีเฉพาะโค้ด CSS เท่านั้น ห้ามมีแท็ก `<style>` รวมอยู่ด้วย
- ไฟล์หรือเพจที่จะนำสไตล์ไปใช้งาน ให้นำเข้าด้วยคำสั่ง `import` ‘./ชื่อไฟล์ CSS’ เช่น สมมติว่าต้องการนำเข้าไฟล์ที่ชื่อ `global.css` ก็กำหนดโค้ดเป็น `import './global.css'`
- สไตล์ที่นำเข้ามาจะมีผลกับแท็ก HTML ที่ตรงกับ Selector ซึ่งระบุไว้ในไฟล์ CSS แม้จะเขียนในแบบ JSX ก็ตาม

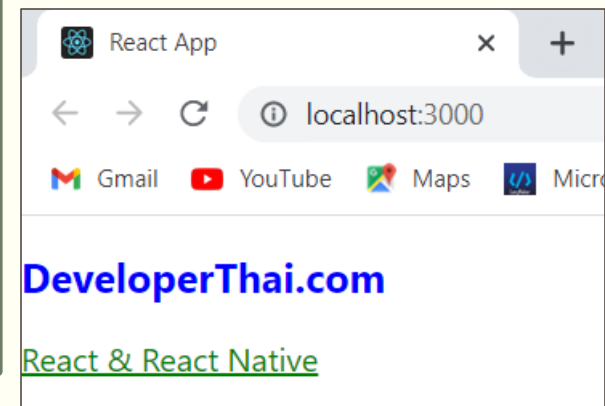
การกำหนดสไตล์ CSS ร่วมกับ JSX

react/app1/src/style.css

```
h2 {  
  font-size: 20px;  
  color: blue;  
}  
  
.title {  
  font-size: 16px;  
  color: green;  
  text-decoration: underline;  
}
```

react/app1/src/App.js

```
import React from 'react'  
import './style.css'  
  
function App() {  
  return (  
    <>  
      <h2>DeveloperThai.com</h2>  
      <div className='title'>React & React Native</div>  
    </>  
  )  
}  
  
export default App;
```



การใช้เมธอด `Array.map()` ร่วมกับ JSX

หากเราต้องการเข้าถึงเพื่อดำเนินการกับสมาชิกทั้งหมดในอาร์เรย์ ซึ่งตามปกติก็มีลูป `for` แบบต่างๆ รวมถึงเมธอด `forEach()` ของอาร์เรย์ให้ใช้งานอยู่แล้ว แต่เนื่องจากลูป `for` รวมถึงเมธอด `forEach()` ไม่คืนค่ากลับมา จึงไม่สะดวกต่อการใช้งานร่วมกับ React JSX อย่างไรก็ตาม อาร์เรย์ใน JavaScript ยังมีเมธอด `map()` ที่ทำงานคล้ายกับ `forEach()` ดังรูปแบบต่อไปนี้

```
map (function(item) {    //หรือ  map(function(item,index) { ... })  
    ...  
    return ...  
})
```

หรืออาจเขียนในแบบ Arrow Function เป็น

```
map ((item) => {    //หรือ map(item => { ... }) หรือ map((item,index) => { ... })  
    ...  
    return ...  
})
```

การใช้เมธอด `Array.map()` ร่วมกับ JSX

- item คือค่าของสมาชิกในการวนลูปแต่ละรอบ
- index อาจระบุเมื่อเราต้องการอ้างถึงลำดับของสมาชิก
- ภายในบล็อกของ `map()` ต้องมีคำสั่ง `return` เพื่อส่งค่าในแต่ละรอบกลับออกไป ยกเว้นกรณีที่ใช้ Arrow Function หากมีเพียงคำสั่งเดียว อาจไม่ระบุ `return` ก็ได้

จะเห็นได้ว่า `map()` จะคล้ายกับ `forEach()` แต่ข้อแตกต่างที่สำคัญคือ เมธอด `map()` จะคืนค่าผลลัพธ์ในแต่ละรอบของการวนลูปกลับคืนมา โดยลักษณะปกติของ `map()` ใน JavaScript นั้น จะคืนค่ากลับมาเป็นอาร์เรย์ของผลลัพธ์ที่ได้ ดังโค้ดต่อไปนี้

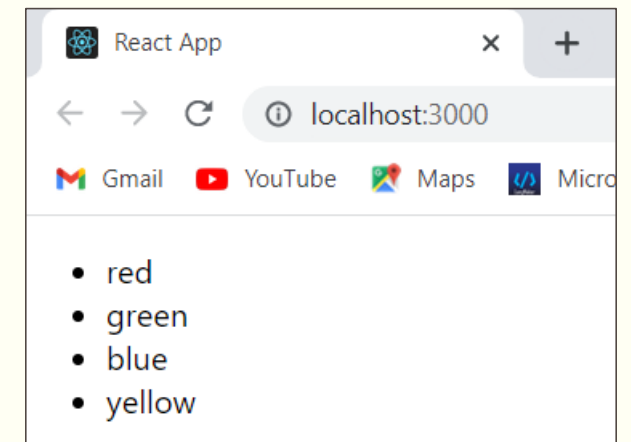
```
let colors = ['red', 'green', 'blue', 'yellow']
let list = colors.map(c => '<li>${c}</li>')
//list = ['<li>red</li>', '<li>green</li>', ...]
```

การใช้เมธอด `Array.map()` ร่วมกับ JSX

แต่เมื่อนำมาใช้กับ JSX อาร์เรย์จะถูกแปลงเป็นสตริงเดียวกันหรือนำสมาชิกแต่ละตัวมาเชื่อมต่อกันนั่นเอง ตามหลักการของ JSX ที่ได้กล่าวไว้แล้ว ซึ่งถือว่าเป็นประโยชน์อย่างมาก เพราะเราไม่ต้องเสียเวลาอ่านค่าสมาชิกมาเชื่อมต่อกันทีละตัว ดังแนวทางในโค้ดต่อไปนี้

```
import React from 'react'
function App() {
  let colors = ['red', 'green', 'blue', 'yellow']
  let list = colors.map(c => <li>{c}</li>) //แทรกค่าแบบ JSX
                                          //list = <li>red</li><li>green</li>...

  return <ul>{list}</ul>
  //หรือเรียกเมธอด map() ใน JSX โดยตรง
  /* return (
    <ul>
      {colors.map(c => <li>{c}</li>)}
    </ul>
  ) */
}
```



การแสดงผลภาพใน React App

รูปภาพ คือองค์ประกอบที่สำคัญอย่างหนึ่งซึ่งเราจำเป็นต้องนำมาแสดงผลในเว็บเพจอยู่บ่อยๆ และสำหรับใน React นั้น ก็มีวิธีการจัดเก็บและอ้างถึงตำแหน่งรูปภาพในแอป ดังนี้

- ตามหลักการแล้ว เราควรจัดเก็บรูปภาพในโฟลเดอร์ public หรืออาจสร้างโฟลเดอร์ย่อยๆ ซ่อนไว้ใน public เพื่อแยกเก็บรูปภาพให้เป็นสัดส่วนจากไฟล์อื่นๆ ก็ได้ เช่น public/images เป็นต้น
- กรณีที่เราจัดเก็บภาพไว้ในโฟลเดอร์ public หากต้องการอ้างถึงภาพในโค้ด JSX ก็อาจเลือกใช้วิธีใดวิธีหนึ่งดังนี้ (สมมติว่าต้องแสดงภาพที่ชื่อ logo192.png ซึ่งอยู่ใน public)

react/app1/src/App.js

```
function App() { //เลือกใช้แบบใดแบบหนึ่ง
  //return 
  //return <img src={"./logo192.png"} alt=""/>
  //return <img src={process.env.PUBLIC_URL + "/logo192.png"} alt=""/>

  //ถ้าจัดเก็บภาพไว้ในโฟลเดอร์ย่อยๆ ซ่อนอยู่ใน public ก็ให้ระบุโฟลเดอร์นั้นด้วย เช่น
  //ถ้าเก็บภาพไว้ใน public/images ก็อ้างอิงตำแหน่งเป็น return 
}
```

การแสดงรูปภาพใน React App

- หรือถ้าเราจัดเก็บภาพในโฟลเดอร์ src การอ้างอิงตำแหน่งภาพในโค้ด JSX ก็ทำดังนี้ (สมมติว่าต้องการแสดงภาพที่ชื่อ LOGO-CIT.png ซึ่งอยู่ในโฟลเดอร์ src)

react/app1/src/App.js

```
import logo from './LOGO-CIT.png' //ภาพอยู่ที่ src/LOGO-CIT.png

function App() {
  return <img src={logo} width="10%" alt="" />
}
```

